The Magic2 deep RNA-sequenecing analyser User Guide

G. Boratyn

D. Thierry-Mieg

J. Thierry-Mieg

National Library of Medicine, National Institutes of Health,
8600 Rockville Pike, Bethesda MD20894, USA.
E-mail: mieg@ncbi.nlm.nih.gov

Abstract

Magic2 is an accurate and sensitive long and short reads RNA deep-sequencing analyser. It can read local fasta/fastq files or import data-sets on the fly from the NCBI short-read-archive (SRA). It can export in a signle pass the alignment files, the coverage plots, the gene expression and many metrics. It is very easy to use because it analyzes in a dry run the top of the sequence files and the hardware to select the optimal strategy. This guide explains how to use this program.

Magic2 is freely avaliable from git@github.com:NLM-DIR/Magic2

Contents

1 Introduction

Considering the fast evolution of the sequencing technologies, we need to accuratelly aligned an ever greater number of short paired-end Illumina reads and ever longer reads from PacBio, Nanopore, Roche and other manufacturers. The files are very large and it is important to minimize the number of steps and intermediate formats.

Magic2 can align local fasta/fastq files but it can also automatically download SRR runs from NCBI short read archives (SRA). Optionally, these data can be cached on the local computer as new fastq files, but if you have a fast network connection to NCBI, this is not mandatory. Converselly, Magic2 is convenient just to download the data that you wish to analyze with other programs.

The program recognizes the eventual adaptors and reports very detailled quality control metrics, It is fast because, in a single pass, it can export the alignments, the gene expression counts, the known and new introns and their support, the polyA addition sites. and a collection of coverage plots indicating the gene expression, the probable breakpoints and the candidate transcriptions starts and stops.

Section 1 gives simple examples of how to use Magic2. Section 2 explains how to download and recompile the program. Section 3 gives all the details on the recognized sequence formats and the way to download the runs from SRA. Section 4 explains the contruction of the target index. The remaining section present the various output files and how to use them.

If available, the code will delegate parts of the calculations to a GPU.

2 Simple examples

First download the code on a Linux or Mac machine with at least 32 Giga bases or RAM. The code will already run if you have 4 processors, and will run faster on a larger machine because it automatically adapts its level of paralellism to the host computer.

2.1 Align data on a raw genome

To just align a set of reads just on a genome, you need to type two successive commands

```
1 magic2 --createIndex IDX.test  -t genome.fasta.gz
2 magic2 -x IDX.test -i read.fasta.gz -o out
```

where 'genome.fasta.gz' is a single fasta file containing the sequence of all the chromosomes and 'read.fasta.gz' contains the reads. The results will be visible in the output directory 'out'. The data files can be gzip compressed or not, the sequences can be given in fasta or fastq format.

To align paired end reads, the pairs can be given in two fasta/fastq files separated by a plus sign, or in interleaved format

```
1 magic2 -x IDX.test -i read_R1.fastq.gz+read_R2.fastq.gz -o out2a
2 magic2 -x IDX.test -i read.sample_12.fastq.gz -o out2b
```

Finally, the data can be downloaded on the fly from the NCBI Short-Read-Archives (SRA) and analyzed simply by providing a set of SRR identifiers

```
magic2 -x IDX.test -srr SRR1234,SRR1235,SRR1236 -o out23
```

Notice that you not need to say if the sequences are DNA or RNA, long or short, or if they contain adaptors. All these properties are automatically recognized by the code by performing a dry run on the first 10 Mega bases, the realigning everything with the propoer strategy.

2.2 Align data on an annotated genome

More accurate and more detailled result will be obtained if you provide a known annotation. Magic2 will favaorize jumping the known introns, and will report the gene expression counts. It will also report the fraction of the sequences mapping on the ribosomal RNA (sometimes as high as 80%) and on the mitochondria. This is important because a complete genome, like the human T2T project, contains very many repetitions of the ribosomal RNAs and echos of parts of the mitochondria, and many programs, including Magic2, would not report highly repetitive alignments.

To provide an annotation, you need to create a small target configuration file called for example T2T.tConfig containing 2 tab separated columns

```
G       T2T.chromosomes.fasta.gz
M       T2T.mitochondria.fasta.gz
R       T2T.rRNA.fasta.gz
I       T2T.gtf.gz
```

where T2T.gff.gz is the T2T annotation file, available for example from NCBI or EBI in gtf or gff format.

Then, create an annotated index by using the parameter (-T) rather then (-t), and align

```
magic2 --createIndex IDX.T2T -T T2T.tConfig

magic2 -x IDX.T2T -srr SRR1234,SRR1235,SRR1236 --full -o out5
```

If you have a very large machine, you could process the 3 runs in parallel, the memory needed by the index will be shared, but still you will consume larger ressources

```
magic2 -x IDX.T2T -srr SRR1234 --full -o out5a &
magic2 -x IDX.T2T -srr SRR1234 --full -o out5b &
magic2 -x IDX.T2T -srr SRR1236 --full -o out5c &
```

In the ouput directory, you will find the alignemnt files, the polyAs, the intron counts and if you used the –full parameter, yo will also find the gene expression counts the canditate transcription starts and ends and a collection of coverage plots.

3 Downloading and installing Magic2

3.1 Simply download the executables

The precompiled executable files can be found at

```
where are the binaries
```

select your machine (Mac or Linux) and eventually also download the GPU enabled code

3.2 Download the source code and recompile

The code is open-source and freely available from github.

```
git clone git@github.com:NLM-DIR/Magic2
```

In case of very intensive use, the precompiled executables may be slightly slower than the programs recompiled locally because the latter may be better taylored to the instruction sets of your actual hardware. To recompile the C code. stay in the download directory and type

```
tcsh
cd ./Magic2
setenv ACEDB_MACHINE LINUX_4_OPT
make libs
cd wsa
make
```

The executable will be the file ./Magic2/bin.LINUX_4_OPT/magic2 Add this directory to your path or copy magic2 in /usr/local/bin so that the command will be always found. You can test if the code run by typing 'magic2 –help' // //

4 Using sequence data available in SRA

4.1 Magic2 can download files from SRA

Magic2 is a convenient tool to download runs from the NCBI short read archives in a format recognized by most bioinformatics program.

Suppose that you are interested in the human paired-end Illumina run SRR24511885. The command

```
magic2 --sraDownload SRR24511885      // creates SRA/SRR24511885.sample_12.fasta.gz
```

will download an interleaved fasta file, in which each pair occupies 4 successive lines and the read identifiers will be s.number/1 and s.number/2. If you are interested in the quality factors, add a parameter –fastq, and if you want to split the pairs into 2 separate files use –split_pairs.

The full syntax of the command is

```
magic2 --sraDownload SRR1,SRR2,SRR3,... [--fastq] [--split_pairs] --maxGB <int>
```

and for example

```
magic2 --sraDownload SRR24511885 --fastq --split_pairs --maxGB 2
// creates SRA/SRR24511885_R1.fastq.gz and SRA/SRR24511885_R2.fastq.gz
```

downloads the first 2 Gigabases of run SRR24511885 and stores it as 2 fastq files. By default, in the absence of the –maxGB parameter, the whole file is downloaded. The number of SRR identifier in the list is not limited. The code will automatically export a single file SRRn.fastq.gz for the single end runs.

4.2 Magic2 can process on the fly SRA runs

If you have a good connection to the NCBI servers, you do not need to store the files locally on your computer. You can align them on the fly with the command

```
magic2 -x IDX --srr SRR1,SRR2,SRR3 -o out_directory
```

magic2 will recognize single-end versus paired end runs and will start aligning as soon as the first few magabases start flowing in. Often, the download may be the limiting factor and all the sequences will be aligned a few seconds after the end of the transfer.

An interesting way to use this command is to evaluate the quality cotrl metrics on the first 2 Gigabases of a number of SRR runs without downloading the files and without exporting the alignments

```
magic2 -x IDX --srr SRR1,SRR2,SRR3 --no_ali -o out_directory
```

This will produce quite fast a detailled portrait of all these runs without consuming any appreciable disk space on your computer.

If the files are already present in the directory ./SRA, for example if they were explicitly downloaded by the previous commands, these cached files will be used rather than accessing the network.

4.3 Magic2 can align and cache the data at the same time

You can combine the previous commands by giving at the same time a list of SRR identifiers a caching request and a target index. For example

```
magic2 --srr SRR24511885 --fastq --sraCaching -x IDX.T2T -o out_dir --full
// creates SRA/SRR24511885.sample_12.fastq.gz
// produces the complete analysis of the run in directory out_dir
```

If the SRR run is already in the ./SRA cache directory in any of the recognized formats, it will not be downloaded a second time.

5 Creation of a target index

As explained in the introduction, a given run can be aligned on a raw genome, but it is often more interesting to compared the reads to an annotated genome. The first step is to construct a taget configuration file

5.1 The target configuration file tConfig

Using a text editor, construct a configuration file describing your target. The format is simple. There are 2 tab separated columns. The first column contains a single letter giving the type, the second column contains a file name.

The recognized types are G, M, C, R, E, A, I, B, V. Several lines can start with the same letter.

G: genome, the fasta file give the genomic sequence of the target, but if more convenient, you can provide each chromosome fasta file on a separate line starting with G

M: Mitochondria, the fasta file of the mitochondria

C: In plants, the sequence of the chloroplast

R: the sequence of the ribosomal RNA precursors. Giving these sequences this is very important because ribosomal RNA may sometimes represent a very high fraction (80and the sequence is repeated many times in the chromosome creating confusion

E: ERCC and other control sequences. Also give the DNA control

A: Adaptors fasta sequence, this file is optional, adaptors can be recognized on the fly

B: bacteria, possible contamination

V: virus, possible contamination

I: Annotation file in gtf or gff format, or a list of introns

In each case the the chromosome name (column 1) must match the G fasta file(s).

```
In gtf/gff format the code expects (at least) 7  columns chrom/method/tag/a1/a2/./[+|-]\n"
tag : [exon] other lines are ignored
a1 a2 : the positions of the first and last base of the exon.
strand : [+|-] indicates the strand, a1 < a2, and the coordinates are 1-based.

In .introns format the code expects 3 columns chrom/a1/a2
a1 a2 : give the positions of the 2 G of the Gt_aG motif
on the top strand a1 < a2, on the bottom strand a1 > a2.
```

Please create the .introns file if the gff/gtf file is not available

5.2 Creating the index

Magic2 belongs to the seed/extend class of aligners. In a first pass, magic2 extract from the genome a set of seeds of fixed length and then searches these seeds in the reads. The length and the density of the genome seeds may be adjusted. By default, magic2 use 16-mers up to 200 Megabases

(drosophila) and 18-mers for longer genomes (human). The default stepping is s=6, meaning we choose the 'best' seed among the next s words. As a result, the maximal distance between seeds is s, and the average distance is s/2, exactly as when using the special seeds refered by Durbin, but in a much simpler way.

The binary files containing the collected seeds are stored in the index directory, together with a summary of the annotaton files. To contruct the index please run the command

```
magic2 --createIndex IDX.target -T tConfig.target [--seedLength <int>] [--step <int>]
```

Changing the default seed length is not recommanded in metazoaires, but it may possibly be interesting to choose shorter seed in a bacteria project. Using a longer stepping like 8 or 10 will produce a faster code, consuming less RAM, but will reduce the sensitivity of the aligner.

6 Alignments

In this project, our main goal is completeness and accuracy. We hope to align the highest possible percentage of reads and of bases in each read, even in the presence of sequencing noise.

To align a run, provide an SRR identifier (it will be downloaded automatically as explained above), or give a fasta or a fastq file. For paired end runs, if they are downloaded from SRA, they are recogbized automatically, other wise you can provide either a pairs of fasta/fastq files separated by a plus sign or provide an interleaved fasta/fastq file which must be called xxx.sample_12.fasta[aq][.gz]

```
magic2 -x IDX.target -i f1.fasta.gz+f2.fasta.gz -o out_dir
```

By default, the alignments are exported in the magic .hits format, but they can also be exported in the magic-blast tabular format or in bam format by providing the parameters

```
magic2 -x IDX.target -i f1.fasta.gz+f2.fasta.gz -o out_dir --hits --tabular --bam
```

If several formats are requested, several files will be exported. The bam format is well known, the .hits and .tabular format give more details, in particular they discribe the mismatches and the introns, and the overhangs more explicitely.

One may also block the exportation of the alignments with the parameter –no_ali

```
magic2 -x IDX.target -i f1.fasta.gz+f2.fasta.gz -o out_dir --no_ali
```

7 Intron support

Our secondary goal is to simplify the user's experience. The program does not require any cost or strategy parameters. It evaluate by itself, via a dry run on the first 10 mega bases, the sequencing technology and the characteristic of the sample and automatically recognizes and clips the eventual

adapters. It also evaluates the hardware to decide the optimal level of parallelisation and if available will run part of the calculations on a GPU.

Thanks to its non-standard parallelization by GO like channels, computing the coverage plots and the gene expression counts come at essentlially no cost, whereas in current methods (HISAT, Star etc.) they usually require a very slow sorting and rescanning of the BAM files. Actually, in Magic2, exporting the BAM alignment files can be considerd optional, since Magic2 may be abale to export directly all the data useful to your project.

A collateral benefit of Magic2 is its speed. The computers, over the last ten to twenty years, have considerably accelerated the computing inits (CPUs), but memory access kept lagging. To compensate, multi-layered cache architectures were designed. The CPUs are grouped in cores, and access the large mamory (the RAM in giga bytes) via several level of caches (typically L1 in kb, L2 in Mb). Considering that memory access is the bottle neck, we designed the Magic2 algotrithms to priviledge sequential memory access. For example

8 Conclusion

Acknowledgments

We would like to thank David Landsman and Tom Madden. This research was supported by the Intramural Research Program of the National Library of Medicine, National Institute of Heath.

References

[1] Richard Durbin and Jean Thierry-Mieg Syntactic Definitions for the ACEDB Database Manager Technical Report: MRC Lab. for Molecular Biology, 1992

[2] Richard Durbin and Jean Thierry-Mieg The acedb genome database, Computational Methods In Genome Research, pages 45-55. Edited by S. Suhai, Plenum Press, New York, 1994

[3] Lincoln D Stein, Jean Thierry-Mieg Scriptable access to the Caenorhabditis elegans genome sequence and other ACEDB databases Genome research, 8,12 pp 1308-1315, 1998

[4] Jean Thierry-Mieg, Danielle Thierry-Mieg and Lincoln Stein ACEDB: The Ace Database Manager In Databases and Systems, 2001 DOI: 10.1007/0-306-46903-0_23

[5] Danielle Thierry-Mieg and Jean Thierry-Mieg AceView: a comprehensive cDNA-supported gene and transcripts annotation. Genome Biology 2006, 7(Suppl 1):S12.

[6] Code and documentation are available from git clone git@github.com:NLM-DIR/Magic2

A Annex

A.1 List of operators and reserved keywords

To summarize, the full list of operators, in their order of precedence is

```
1   ; TITLE order\_by
2   from select , where
3   in := =
4   || OR ^^ XOR && AND ! NOT
5   like =~ ~ == != >= <= > < >~ <~
6   ISA
7   modulo + - * / ^
8   DNA PEPTIDE PEP CODING DATEDIFF
9   COUNT MIN MAX SUM AVERAGE STDDEV
10  .class .name .timestamp
11  >> -> # =>
12  @ OBJECT
13  () {} []
```

with the restriction that the 3 kinds of parentheses and also the quotes must be nested correctly (["bb"]) is correct but (x[y)] is illegal. Operators written in capitals, i.e. DNA or AVERAGE, must be written in capitals (are case-sensitive)

A.2 History of the development of the acedb query language

The query syntax described in this document results from the convergence of several developments - the original acedb query language of 1990, available in the 'query' box of the acedb graphic interface - the table maker system of 1992, with its user friendly graphic interface - the original AQL query language developed at the Sanger around 2002

The general idea was to learn from the 3 existing systems, conserve the best aspects of each, fix their limitations and clean up the interface. For example,

– The acedb query language is very terse and expressive, but it is limited to constructing sets of objects and does not allow to display the content of selected fields.

– The table maker was meant to fix these limitations. Using its rich graphic interface, one can easily construct a tables involving objects, numbers, texts and even DNA sequences. However, even a simple tables cannot be constructed on the command line. One need always to construct the table graphically, to save its definition in a definition file, and run the query using that file.

– The original AQL query language was ment to fix this limitation. AQL defined a command line syntax, reminiscent of SQL, but adapted to the idiosyncrasies of an object oriented database like acedb. It made it in principle possible to compose a table on the command line or using a library call. Unfortunately, the syntax was slightly too rich, encouraging the user to write convoluted queries, and always verbose. Since there was no graphic interface to help compose a valid query,

and since the original AQL compiler did not explain the eventual errors, it was hard to write a non-trivial valid query. Finally, AQL execution was slower than table-maker, and lacked access to DNA and protein sequences.

From the user point of view, the previous interfaces are maintained, but the old style queries are translated internally and executed by the new query engine. In particular, the graphic table-maker interface remains valid and can still be used to construct complex queries and most original AQL queries remain valid. In practice the shortcuts defined in section 2.5 map the terse query language of the acedb graphic interface, first released in 1990, into a (modification of) the full fledged AQL queries developed around 2002. As a result, these 2 types of queries, and the table constructed via the acedb graphic table maker interface, are all expressed in the new unified syntax and the database exploration can be executed using a single highly optimized query engine.

A.3 Code availability

The acedb source code can be downloaded from

https : *//github.com/ncbi/AceView*, *https* : *//github.com/jtmieg/AceView*

The tar ball is effectively identical to the distribution of the AceView/Magic pipeline [**?**] which uses acedb as the underlying database, and can organize the workflow and manage the meta-data of tens of thousands of next-generation sequencing runs. The same package underlies the Gene annotation public web site https://www.ncbi.nlm.nih.gov/AceView [5]

The program is written in C and has very few external dependencies. To compile the code try

```
tcsh
gunzip -c magic.*.tar.gz | tar xf -
setenv ACEDB_MACHINE LINUX_4_OPT
cd magic*
make libs tace
make -k all
```

The code can also compile on the Mac using setenv ACEDB_MACHINE MAC_X_64_OPT or on any other linux/unix platform. Multiple choices are offered in the directory wmake. The tace command-line interface presented in the present document should compile without difficulty. The xace graphic-interface requires the installation of the public X11 development environment as explained in the README file situated at the top of the acedb distribution.

A test-bench containing examples of queries can be constructed using the command

```
tcsh wbql/bqltest.tcsh
```

After running the script, the file ACEDB_QUERY_LANGUAGE_TEST/test.out contains the output of the commands contained in test.cmd and the file test.diff, which contains the differences between test.out and test.out.expected, should be empty. The database can be examined and further queries can be tested using the command

```
bin*/tace ACEDB_QUERY_LANGUAGE_TEST
```

which will activate the command line interface of acedb. All the examples in the present document can be copied in the interface and should give proper answers. A question mark will invoke the on-line help and list all the possible commands. In particuler, the acedb command 'show' will display the full content of the active objects and will help to understand why they where selected by the query.

Please try the code and send us feedback.

A.4 Running an acedb session

As explained above, a small test database can be constructed automatically. More generally, the directory waligner/metadata contains some examples of large acedb schemas. Once you have constructed a local directory containing an empty sub-directory database and a sub-directory wspec defining the schema and a data file xxx.ace, you can initialize a database and parse an ace file as follows:

```
bin*/tace .  // launch the compiled text-acebd on the local directory
y            // yes, initialize the database
  parse myfile.ace              // parse some data
  save  // save the data in compiled binary format
  quit
```

Acedb can be regarded as a data compiler. It can read successively several data files, merge them and compile them into a compact binary format allowing fast retrieval. Clear messages signal places in the data file which do not conform to the schema. You can fix the file and reload it. Reading the same file several times is idempotent, i.e. it does not cause any problem.

Subclasses are configured in the file wspec/subclasses.wm

```
  Class Prolific_author       // The class name must start with a letter
  Visible                     // Make it available in graphic menus
  Is_a_subclass_of Person     // Inherit from class Person
  Filter "COUNT Papers >= 5"  // Chosen filter
```

One can then use the database and ask queries on the command line or import it from a file using the $-i$ parameter or redirect the output to a file using $-o$

```
bin*/tace . // start the system
   select ...  from ... where // a query
xx yy zz      // the reply comes on stdout
xx yy zz
   select -o f.out ... from ... where ...
              // the reply goes in the file f.out
   select -a ... // the reply is quote protected (.ace format)
```

```
select -i f.bql ... // file f.bql contains the query
select -i f.bql -s  // silent mode
select @ ... // use the new active set
? select  // help of the command line parameters
?         // list all the acedb commands
quit
```

In silent mode (with semi column at the end of the query, or using the option -s (-silent)) the output is suppressed but the active list is modified.

Acedb contains dozens of commands available by typing ? or ?command. There is also a rich graphic interface, available using the command *bin.* $*$ */xace.*, which generates on the fly in HTML5/SVG language all the plots and diagrams presented on the AceView server [5]. It provides among other tools a graphic interface helping to compose complex tables. But the description of the graphic acedb system is out of the scope of this user guide.

The MAGIC pipeline [**?**] is a good example of a complex system relying on acedb.

A.5 C language API

The acedb system has a C language API. The application code can either be linked into the open-source database server code, in which case it directly shares the caches of the database engine, or it can be part of a client code which calls the server via tcp. The interesting point is that exactly the same code works in both situations, one changes from a server to a client code by simply changing the library used at link stage. Here is an example of a complete C program invoking the query language

```
#include <ac.h>  /* acedb C language API header */

int main (int argc, const char **argv)
{
  AC_HANDLE h = ac_new_handle () ;
  const char *errors = 0 ;
  const char *dbNameS = "acedb_directory" ;              // server case
  const char *dbNameC = "t:machine_name:port_number" ;   // client case
  AC_DB db = ac_open_db (dbNameX, &errors);              // X = S|C
  const char *query = "select p in class person where p like a*" ;
  AC_KEYSET aks = 0 ;  /* initial keyset, populates the @ symbol */
  AC_TABLE t = ac_bql_table (db, query, aks, errors, h) ;

  printf ("Found % lines in the table\n", t->rows) ;

  ac_db_close (db) ;
  ac_free (h) ;
  return 0 ;
} /* main */
```

The interface is detailled in the self documented header file wh/ac.h.

All comments are welcome.